



Manual de Apoyo para Pruebas de Caja Negra y Blanca, CASOS DE USO

TexFashion

09-05-2025

Bogotá DC

1. Login

Caso de Uso

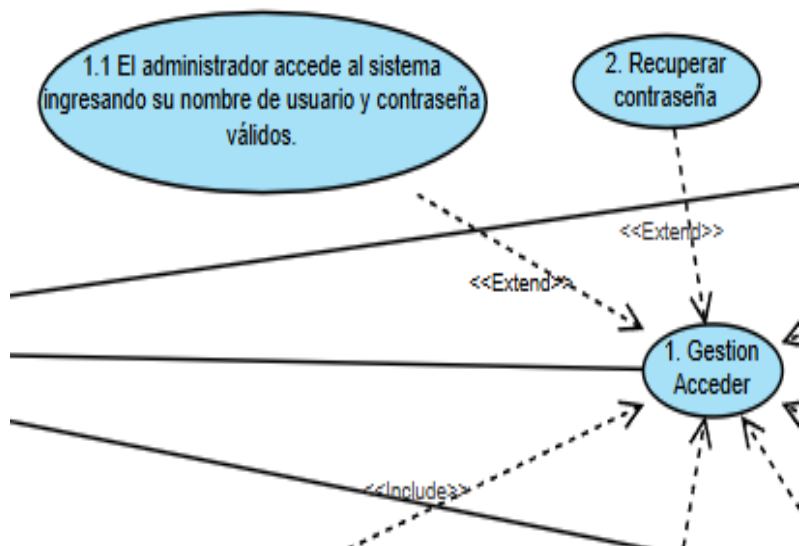
Este caso de uso describe cómo un usuario accede al sistema proporcionando credenciales válidas (nombre de usuario y contraseña). El sistema verifica la validez de los datos ingresados y, si son correctos, concede el acceso al sistema.

Precondiciones:

- El usuario debe estar registrado en el sistema.
- El sistema debe estar disponible.

Flujo Principal (Escenario Exitoso):

1. El usuario accede a la pantalla de inicio de sesión.
2. El sistema solicita nombre de usuario y contraseña.
3. El usuario introduce su nombre de usuario y contraseña.
4. El sistema verifica las credenciales ingresadas.



5. Si las credenciales son válidas, el sistema concede acceso al usuario y lo redirige a la página principal.

Código

```
public function validateUser($data)
{
    try {
        // Encriptar la contraseña ingresada con MD5
        $hashedPassword = md5($data['password']); // Asegúrate de que el campo del
formulario sea 'password'

        //consulta para validar si el usuario existe o no
        $strSql = "SELECT id, correo_electronico, contrasena AS encrypted_password, rol
FROM usuario WHERE correo_electronico = '{$data['email']}'";

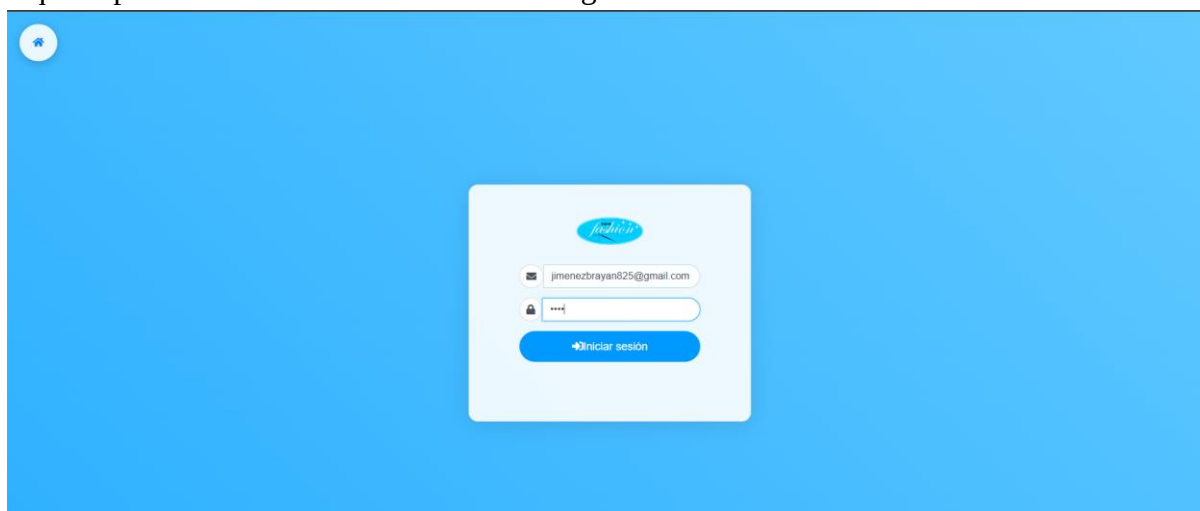
        //se ejecuta la consulta se pasa como parametro la consulta
        $query = $this->pdo->select($strSql);

        //validacion
        //si existe el id
        if (isset($query[0]->id)) {
            // Validar si la contraseña coincide
            if ($query[0]->encrypted_password === $hashedPassword) {
                //si lo encuentra me retorna un verdadero
                return $query[0];
            } else {
                return 'Error al Iniciar Sesión. Verifique sus Credenciales';
            }
        } else {
            return 'Error al Iniciar Sesión. usuario no existe';
        } //si es falso reorna error

    } catch (PDOException $e) {
        return $e->getMessage();
    }
}
```

Img Interfaz:

Aquí se puede visualizar el Front End del Login



2. Gestion Usuario

Caso de uso

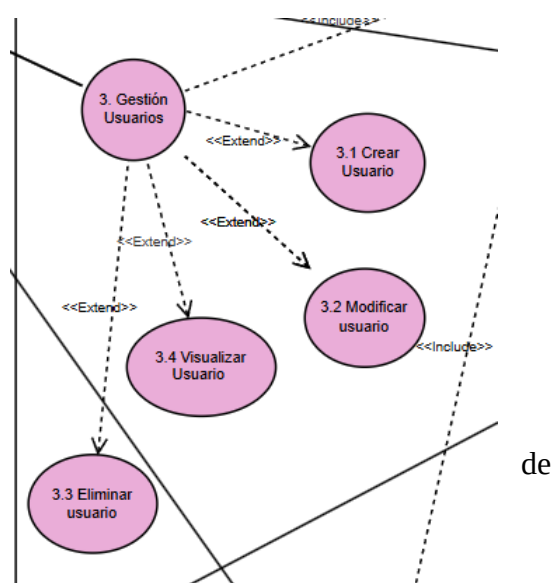
Este caso de uso describe cómo el administrador gestiona las cuentas de los usuarios del sistema. Incluye la creación de nuevos usuarios, la visualización de la lista de usuarios, la modificación de información existente y la eliminación de usuarios. Estas acciones solo están disponibles para usuarios con privilegios de administrador.

Precondiciones:

- El administrador debe haber iniciado sesión en el sistema.
- El sistema debe estar operativo.

Flujo Principal (Escenario Exitoso):

1. El administrador accede a la sección de gestión de usuarios.
2. El sistema muestra la lista de usuarios registrados.
3. El administrador puede realizar las siguientes acciones:
 - **Crear** un nuevo usuario, ingresando los datos requeridos.
 - **Leer** o consultar la información un usuario.
 - **Actualizar** los datos de un usuario existente.
 - **Eliminar** un usuario del sistema.



4. El sistema confirma la acción y actualiza la base de datos en consecuencia.

Código

```
class Usuarios
{
```

```
private $pdo;

public function __construct()
{
    try {
        $this->pdo = new Database;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function getAll()
{
    try {
        $sql = "SELECT u.*, r.Rol, d.TipoDocumento
        FROM usuario u
        JOIN rol r ON u.rol = r.idRol
        JOIN documento d ON u.tipo_documento = d.IdDocumento";
        return $this->pdo->select($sql);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function getUsuariosId($id)
{
    try {
        $sql = "SELECT * FROM usuario WHERE id = :id";
        return $this->pdo->select($sql, ['id' => $id]);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function newUsuarios($data)
{
    try {
```

```
$this->pdo->insert('usuario', $data);
return true;
} catch (PDOException $e) {
    return $e->getMessage();
}
}

public function editUsuarios($data)
{
    try {
        $where = 'id = ' . $data['id'];
        return $this->pdo->update('usuario', $data, $where);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function deleteUsuarios($id)
{
    try {
        $where = 'id = ' . intval($id);
        return $this->pdo->delete('usuario', $where);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function obtenerCorreoPorId($id)
{
    try {
        $sql = "SELECT correo_electronico FROM usuario WHERE id = ?";
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([$id]);
        return $stmt->fetch(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        return null;
    }
}
```

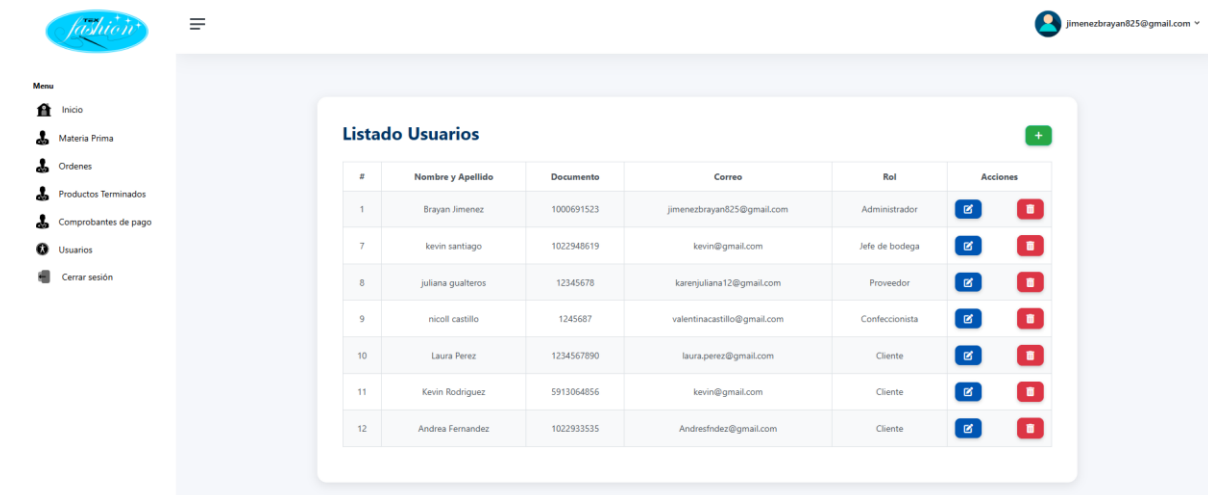
```

}

// Este es el método que estabas intentando agregar
public function getTAll()
{
    try {
        $sql = "SELECT id, nombre, apellido FROM usuario WHERE rol = 4 AND status =
'IN'";
        return $this->pdo->select($sql);
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}
}

```

Img Interfaz



The screenshot displays the 'Listado Usuarios' (Users List) interface. On the left is a sidebar menu with options: Inicio, Materia Prima, Ordenes, Productos Terminados, Comprobantes de pago, Usuarios, and Cerrar sesión. The main area shows a table with the following data:

#	Nombre y Apellido	Documento	Correo	Rol	Acciones
1	Brayan Jimenez	1000691523	jimenezbrayan825@gmail.com	Administrador	 
7	kevin santiago	1022948619	kevin@gmail.com	Jefe de bodega	 
8	juliana gualteros	12345678	karenjuliana12@gmail.com	Proveedor	 
9	nicoli castillo	1245687	valentinacastillo@gmail.com	Confeccionista	 
10	Laura Perez	1234567890	laura.perez@gmail.com	Cliente	 
11	Kevin Rodriguez	5913064856	kevin@gmail.com	Cliente	 
12	Andrea Fernandez	1022933535	Andresfhdez@gmail.com	Cliente	 

Modificar

Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Modificar Usuario

Nombres	Apellidos
Brayan	Jimenez
Tipo Documento	Documento
Cedula Ciudadania	1000691523
Correo Electrónico	Dirección
jimenezbrayan825@gmail.com	KR 6F ESTE #89C 48 SUR
Teléfono	Fecha de Nacimiento
3194753477	20/01/2002
Rol	
Administrador	

Actualizar

Crear

Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Crear Nuevo Usuario

Nombres	Apellidos
Tipo Documento	Documento
Seleccione...	
Correo Electrónico	Dirección
Teléfono	Fecha de Nacimiento
	dd/mm/aaaa
Rol	
Seleccione...	

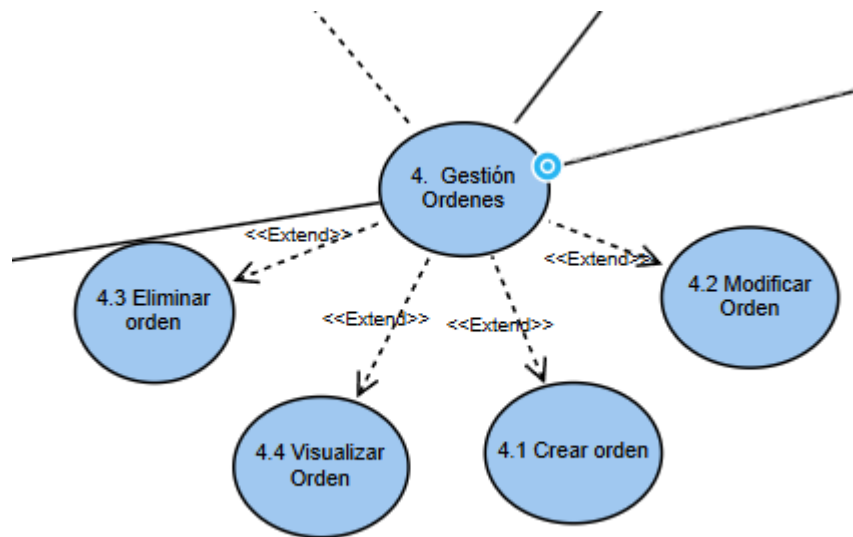
Guardar

3. Gestión orden

Este caso de uso describe cómo los diferentes actores (Administrador, Jefe de Bodega, Proveedor y Confeccionista) pueden gestionar las órdenes de trabajo mediante funciones CRUD (Crear, Leer, Actualizar, Eliminar). El sistema también proporciona funcionalidades de notificaciones automáticas y un calendario para visualizar las órdenes próximas a su fecha de entrega.

Precondiciones:

- El usuario debe haber iniciado sesión en el sistema y tener uno de los roles autorizados.
- El sistema debe estar disponible y sincronizado con la base de datos de órdenes.



Flujo Principal (Escenario Exitoso):

1. El usuario accede a la sección de Gestión de Órdenes.
2. El sistema muestra la lista de órdenes existentes con detalles (fecha, estado, responsable, etc.).
3. El usuario puede:
 - **Crear** una nueva orden ingresando información como descripción, fecha de entrega, materiales, estado, etc.
 - **Consultar (Leer)** el detalle de una orden existente.
 - **Actualizar** el estado o los datos de una orden (por ejemplo, cambiar la fecha de entrega, asignar responsable).
 - **Eliminar** una orden (según permisos).
4. El sistema actualiza la información de la orden en la base de datos.
5. El sistema:

- Envía **notificaciones automáticas** a los actores involucrados en la orden cuando hay cambios relevantes (nuevas asignaciones, modificaciones, fechas próximas).
- Muestra un **calendario** con las fechas de entrega próximas, destacando las más cercanas.

Código:

```
class Orden
{
    private $idOrden;
    private $idCliente;
    private $Fecha_Orden;
    private $Total_Total;
    private $Cantidad_Producto;
    private $Fecha_Entrega;
    private $idProductosTerminados;
    private $idMateriaPrima;
    private $Estado;
    private $status;
    private $pdo;

    function __construct()
    {
        try {
            $this->pdo = new Database;
        } catch (PDOException $e) {
            die($e->getMessage());
        }
    }

    public function getAll()
    {
        try {
            $strSql = "SELECT * FROM orden o JOIN estados e ON o.Estado = e.idEstados
                        JOIN productos_terminados pt ON o.idProductosTerminados = pt.idProductos
                        JOIN materia_prima mp ON o.idMateriaPrima = mp.idProducto
                        JOIN usuario u ON o.idCliente = u.id WHERE o.status = 'IN'";
```

```

        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function newOrdenes($data)
{
    try {
        $this->pdo->insert('orden', $data);
        return true;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getOrdenById($idOrden)
{
    try {
        $strSql = "SELECT o.*, e.Estados AS EstadoNombre, pt.Nombre_Producto AS
ProductoTerminadoNombre, mp.Nombre AS MateriaPrimaNombre, c.nombre AS
ClienteNombre, c.apellido AS ClienteApellido
        FROM orden o
        JOIN estados e ON o.Estado = e.idEstados
        JOIN productos_terminados pt ON o.idProductosTerminados =
pt.idmateria_prima
        JOIN materia_prima mp ON o.idMateriaPrima = mp.idProducto
        JOIN usuario c ON o.idCliente = c.id
        WHERE o.idOrden = :idOrden";

        $arrayData = ['idOrden' => $idOrden];
        $query = $this->pdo->select($strSql, $arrayData);
        return $query;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

```

```

public function editOrdenes($data)
{
    try {
        $strWhere = 'idOrden = ' . $data['idOrden'];
        $this->pdo->update('orden', $data, $strWhere);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function deleteOrdenes($idOrden)
{
    try {
        // Actualiza el estado de la orden a 'OUT' (en lugar de eliminarla físicamente)
        $strWhere = 'idOrden = :idOrden';
        $data = ['status' => 'OUT', 'idOrden' => $idOrden];

        // Realiza la actualización
        $this->pdo->update('orden', $data, $strWhere);

        return true;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getRol()
{
    try {
        $strSql = "SELECT idRol, rol FROM rol WHERE idRol = 4";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

```

```
}

public function getProductosTerminados()
{
    try {
        $strSql = "SELECT idProductos, Nombre_Producto FROM productos_terminados";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getMateriasPrimas()
{
    try {
        $strSql = "SELECT idMateriaPrima, NombreMateriaPrima FROM materia_prima";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getEstados()
{
    try {
        $strSql = "SELECT idEstados, Estados FROM estados";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}
}
```

Img Interfaz:

Menu

- Inicio
- Materia Prima
- Ordenes
- Productos Terminados
- Comprobantes de pago
- Usuarios
- Cerrar sesión

jimenezbrayan825@gmail.com

Listado de Órdenes

ID Orden	Cliente	Fecha Orden	Total	Cantidad Producto	Fecha Entrega	Producto Terminado	Materia Prima	Estado	Acciones
13	Laura Perez	2024-11-30	50.00	15	2025-03-19	Cofas4	Telaa	Habilitada	
14	Laura Perez	2024-11-30	50.00	15	2025-03-19	Cofas4	Telaa	Habilitada	
1	Juliana gualteros	2024-11-30	154000.00	344	2024-11-30	Cofas4	paco	No_Habilitada	
9	Laura Perez	2024-11-14	150.00	2	2023-10-20	Cofas4	Telaa	No_Habilitada	
15	Andrea Fernandez	2024-12-04	25000.00	3	2024-12-06	ChaquetaPromm	Telaa	No_Habilitada	

Crear Orden

Menu

- Inicio
- Materia Prima
- Ordenes
- Productos Terminados
- Comprobantes de pago
- Usuarios
- Cerrar sesión

jimenezbrayan825@gmail.com

Crear Nueva Orden

Cliente

Selecione...

Fecha Orden

dd/mm/aaaa

Total

Cantidad Producto

Fecha Entrega

dd/mm/aaaa

Producto Terminado

Selecione...

Materia Prima

Selecione...

Estado

Selecione...

Guardar

Modificar Orden



Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Editar Orden

Cliente	Laura Perez
Fecha Orden	dd/mm/aaaa
Total	50.00
Cantidad Producto	15
Fecha Entrega	11/05/2025
Producto Terminado	Cofas4
Materia Prima	Seleccione...
Estado	Habilitada
<button>Guardar</button>	

Notificación de órdenes



Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

May 2025						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
Entrega prima de Ode						
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Listado de Órdenes



Notificaciones de Órdenes Próximas a Cumplirse

Orden #13: La orden está por entregarse el 2025-05-11. Cliente: Cliente desconocido. Estado: Estado desconocido. Total: \$50.00. Cantidad de Productos: 15

4. Gestión Materia Prima

Este caso de uso permite a los actores autorizados realizar la gestión completa de la materia prima necesaria para el funcionamiento del sistema. Incluye las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre los registros de materia prima, tales como telas, hilos, insumos, etc. La información gestionada es clave para la planificación y ejecución de las órdenes de producción.

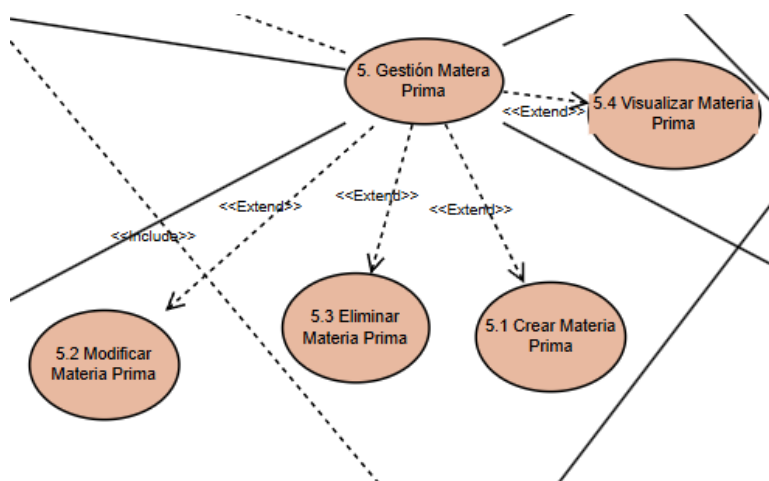
Precondiciones:

- El usuario debe haber iniciado sesión en el sistema con uno de los roles habilitados.
- El sistema debe estar conectado con la base de datos de inventario de materia prima.

Flujo Principal (Escenario Exitoso):

1. El usuario accede a la sección de Gestión de Materia Prima.
2. El sistema muestra la lista de materiales existentes con su información (tipo, cantidad, ubicación, proveedor, fecha de ingreso).
3. El usuario puede:

- **Crear** un nuevo registro de materia prima (nuevo insumo o lote).
- **Consultar** los detalles de un material (por nombre, tipo, proveedor, etc.).
- **Actualizar** datos como cantidad disponible, ubicación o proveedor asociado.
- **Eliminar** un registro de materia prima (según permisos y condiciones).



4. El sistema guarda los cambios en la base de datos y confirma la acción.

Código:

```

class MateriaPrima
{
    private $IdProducto;
    private $Nombre;
    private $Descripcion;
    private $Fecha_Ingreso;
    private $Precio_Unidad;
    private $Cantidad_Stock;
    private $Id_Proveedor;
    private $Categoria;
    private $Unidad_Medida;
    private $Fecha_Actualizacion;
    private $Estado;
    private $status;
    private $pdo;

    function __construct()
    {
        try {
            $this->pdo = new Database;
        } catch (PDOException $e) {
            die($e->getMessage());
        }
    }

    public function getAll()
    {
        try {
            $strSql = "SELECT mp.*, e.Estados AS EstadoNombre, c.Categoria AS
CategoriaNombre, um.Uni_Med AS UnidadMedidaNombre,
            u.nombre AS ProveedorNombre, u.apellido AS ProveedorApellido FROM
materia_prima mp JOIN estados e ON mp.Estado = e.idEstados
            JOIN categorias c ON mp.Categoria = c.idCategoria JOIN unidadmedida um ON
mp.Unidad_Medida = um.MedidaId
            JOIN usuario u ON mp.id_Proveedor = u.id WHERE MP.status = 'IN'";

```

```

        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function newMateriaPrima($data)
{
    try {
        $this->pdo->insert('materia_prima', $data);
        return true;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getMateriaPrimaId($idProducto)
{
    try {
        $strSql = "SELECT MP.*, E.Estados AS EstadoNombre, C.Categoria AS
CategoriaNombre, UM.Uni_Med AS UnidadMedidaNombre, U.nombre AS
ProveedorNombre, U.apellido AS ProveedorApellido FROM materia_prima AS MP INNER
JOIN categorias AS C ON C.idCategoria = MP.Categoria INNER JOIN unidadmedida AS
UM ON UM.MedidaID = MP.Unidad_Medida INNER JOIN estados AS E ON E.idEstados =
MP.Estado INNER JOIN usuario AS U ON U.id = MP.id_Proveedor WHERE
MP.idProducto = :idProducto ";
        $arrayData = ['idProducto' => $idProducto];
        $query = $this->pdo->select($strSql, $arrayData);
        return $query;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function editMateriaPrima($data)
{

```

```

try {
    $strWhere = 'idProducto = ' . $data['idProducto'];
    $this->pdo->update('materia_prima', $data, $strWhere);
} catch (PDOException $e) {
    die($e->getMessage());
}
}

public function deleteMateriaPrima($idProducto)
{
    try {
        // Cambia la consulta para que se ajuste al método que estás usando
        $strWhere = 'idProducto = :idProducto';
        $data = ['status' => 'OUT', 'idProducto' => $idProducto];

        // Asegúrate de que tu método update esté manejando los parámetros correctamente
        $this->pdo->update('materia_prima', $data, $strWhere);

        return true; // Indica que la operación fue exitosa
    } catch (PDOException $e) {
        return $e->getMessage(); // Devuelve el mensaje de error
    }
}

public function getProveedores($Proveedor)
{
    try {
        $strSql = "SELECT id, nombre, apellido FROM usuario WHERE Rol = 5";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

```

```
public function getCategorias($Categoria)
{
    try {
        $strSql = "SELECT idCategoria, Categoria FROM categorias";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getEstados($Estado)
{
    try {
        $strSql = "SELECT idEstados, Estados FROM estados";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getUnidadMedida($UniMed)
{
    try {
        $strSql = "SELECT MedidaID, Uni_Med FROM unidadmedida";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}
}
```

Img Interfaz

Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Listado Materia Prima

ID Producto	Nombre	Descripción	Fecha Ingreso	Precio Unidad	Cantidad Stock	Proveedor	Categoría	Unidad Medida	Fecha Actualización	Estado	Acciones
6	Telaa	asdasdasdas	2024-11-30	150000.00	150	juliana gualteros	Algodón	Sacos	2024-11-20	Disponible	 
8	Hilo	hilo amarillo	2024-11-30	150.00	15	juliana gualteros	Algodón	Bobinas	2024-11-30	Disponible	 
5	paco	l	2024-11-28	15000.00	14	juliana gualteros	Lana	Sacos	2024-11-21	No_Disponible	 
4	medias	larga de futbol	2024-11-30	123344.00	12	juliana gualteros	Seda	Tex	2024-11-30	Habilitada	 
10	Cremalleras	Cremalleras color plateado	2024-11-30	15000.00	5	juliana gualteros	Lino	Metros	2024-11-30	No_Habilitada	 

Crear Materia Prima

Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Crear Nuevo Producto

Nombre

Descripción

Fecha Ingreso

Precio Unidad

Cantidad Stock

Proveedor

Categoría

Unidad de Medida

Fecha Actualización

Estado

Guardar

Modificar Materia Prima



Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Editar Producto

Nombre	Tela
Descripción	asdasdsadas
Fecha Ingreso	30/11/2024
Precio Unidad	150000.00
Cantidad Stock	150
Proveedor	juliana guateros
Categoría	Algodón
Unidad de Medida	Sacos
Fecha Actualización	20/11/2024
Estado	Disponible
Actualizar	

5.. Gestión Comprobante Pago

Este caso de uso permite al Administrador y al Proveedor gestionar los comprobantes de pago relacionados con las transacciones del sistema. Incluye la capacidad de cargar, consultar, actualizar y eliminar comprobantes, así como verificar su validez. Este proceso garantiza la trazabilidad y control de los pagos realizados por servicios o entrega de materiales.

Precondiciones:

- El usuario debe haber iniciado sesión como Administrador o Proveedor.
- Debe existir una transacción o pedido válido asociado al comprobante.
- El sistema debe tener conexión con el módulo de finanzas o pagos.

Flujo Principal (Escenario Exitoso):

1. El usuario accede a la sección de Comprobantes de Pago.

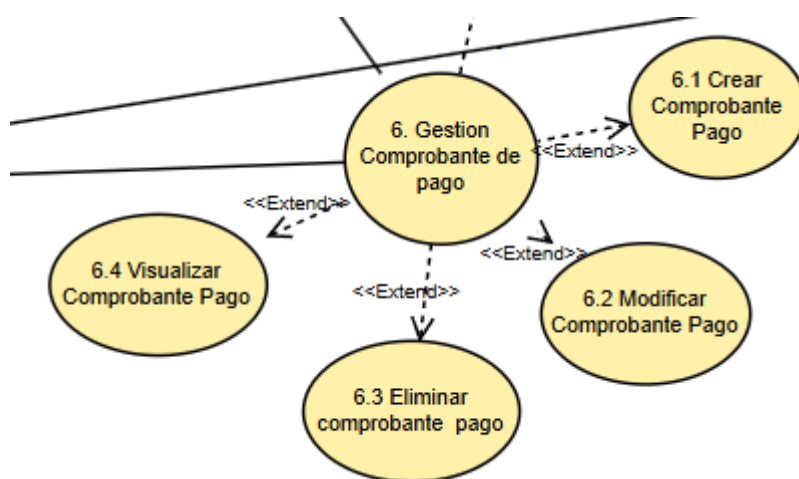
2. El sistema muestra los comprobantes disponibles o las opciones para gestionarlos.

3. El usuario puede realizar una o varias de las siguientes acciones:

- Crear/Cargar un nuevo comprobante de pago (adjuntar archivo, ingresar datos como número, fecha, monto, entidad bancaria).
- Consultar comprobantes anteriores relacionados con una orden o proveedor.
- Actualizar información del comprobante (por ejemplo, corregir un dato o cambiar archivo adjunto).
- Eliminar un comprobante (según permisos y condiciones).

4. El sistema valida la acción y actualiza la base de datos.

5. Se puede notificar al otro actor implicado cuando un comprobante es cargado o modificado.



Código:

```
class Factura
{
    private $idFacturas;
    private $Cantidad;
    private $informacion_del_producto;
    private $Fecha_de_Emision;
    private $Precio_Total;
    private $Numero_Factura;
    private $idCliente ;
    private $Direccion_Facturacion;
    private $Estado_Factura ;
    private $Fecha_Pago;
    private $Referencia_Pago;
    private $status;
    private $pdo;

    public function __construct()
    {
        try {
            $this->pdo = new Database();
        } catch (PDOException $e) {
            die($e->getMessage());
        }
    }

    // Obtener todas las facturas
    public function getAll()
    {
        try {
            $strSql = "SELECT f.*,e.Estados AS EstadoNombre,u.nombre, u.apellido,
pt.Nombre_Producto
                        FROM facturas f
                        JOIN estados e ON f.Estado_Factura = e.idEstados
                        JOIN usuario u ON f.idCliente = u.id
```

```

        JOIN productos_terminados pt ON f.Informacion_del_Producto =
pt.idProductos WHERE f.status = 'IN';
        return $this->pdo->select($strSql);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

// Crear nueva factura
public function newFactura($data)
{
    try {
        $this->pdo->insert('facturas', $data);
        return true;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

// Obtener factura por ID
public function getFacturasId($idFacturas)
{
    try {
        $strSql = "SELECT f.*,
                        e.Estados AS EstadoNombre,
                        u.nombre,
                        u.apellido
                    FROM facturas f
                    JOIN estados e ON f.Estado_Factura = e.idEstados
                    JOIN usuario u ON f.idCliente = u.id WHERE f.idFacturas = :idFacturas ";
        $params = ['idFacturas' => $idFacturas];
        return $this->pdo->select($strSql, $params);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

```

```
// Editar factura
public function editFactura($data)
{
    try {
        $strWhere = 'idFacturas = ' . $data['idFacturas'];
        $this->pdo->update('Facturas', $data, $strWhere);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

// Eliminar factura (cambia el estado a inactivo)
public function deleteFacturas($idFacturas)
{
    try {
        // Cambia la consulta para que se ajuste al método que estás usando
        $strWhere = 'idFacturas = :idFacturas';
        $data = ['status' => 'OUT', 'idFacturas' => $idFacturas];

        // Asegúrate de que tu método update esté manejando los parámetros correctamente
        $this->pdo->update('Facturas', $data, $strWhere);

        return true; // Indica que la operación fue exitosa
    } catch (PDOException $e) {
        return $e->getMessage(); // Devuelve el mensaje de error
    }
}

// Obtener todos los estados

public function getEstados($Estado)
{
    try {
        $strSql = "SELECT idEstados, Estados FROM estados";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
```

```

        return $e->getMessage();
    }
}

public function getProductosT()
{
    try {
        $strSql = "SELECT * FROM productos_terminados";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

// Obtener todos los clientes
public function getUsuariosPTAll()
{
    try {
        $strSql = "SELECT id, CONCAT(nombre, ' ', apellido) AS NombreCompleto FROM
usuario WHERE rol = 4";
        return $this->pdo->select($strSql);
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}
}

```

Img Interfaz:

Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Comprobante de pago

idFacturas	Cantidad	Info producto	Fecha de emision	Precio total	Numero factura	Cliente	Lugar de factura	Estado de Factura	Fecha de Pago	Referencia de Pago	Acciones
100	3	Cofias4	2024-11-30	12312.00	r1232	Laura Perez	Camera 7c #90	No_Habilitada	2024-11-30	r232	 
102	956	Cofias4	2024-11-15	150000.00	15	Laura Perez	2024-11-15	No_Habilitada	2024-11-21	1520	 
104	-98	Cofias4	2024-11-08	-150.00	18	Laura Perez	2024-11-28	No_Habilitada	2024-11-20	23	 
105	-98	Cofias4	2024-10-30	150.00	12	Laura Perez	2024-11-27	No_Habilitada	2024-11-05	23	 
101	14	Uniforme Vigilante	2024-11-08	1500.00	12	kevin santiago	cra 43 sur 56	Pagado	2024-11-15	r231	 
106	95	Uniforme enfermeria	2024-11-30	150000.00	f_22	Laura Perez	Camera 7c #90	Pagado	2024-11-30	23	 

Crear Comprobante pago

Menu

-  Inicio
-  Materia Prima
-  Ordenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Crear Comprobante de pago

Cantidad

Info de Producto
 Seleccione...

Fecha de Emision
 dd/mm/aaaa

Precio Total

Numero Factura

Lugar de Facturación
 Seleccione...

Cliente
 Seleccione...

Estado
 Seleccione...

Fecha de Pago
 dd/mm/aaaa

Referencia de Pago

Modificar comprobante de pago



Menu

-  Inicio
-  Materia Prima
-  Órdenes
-  Productos Terminados
-  Comprobantes de pago
-  Usuarios
-  Cerrar sesión

Actualizar Comprobante de pago

Cantidad	3
Producto Terminado	Seleccione...
Fecha Emision	30/11/2024
Precio Total	12312.00
Numero Factura	r1232
Lugar De Facturación	Carrera 7c #90
Cliente	Laura Perez
Estado	No_Habilitada
Fecha de pago	30/11/2024
Referencia de factura	r232
Actualizar	

6.. Gestión Productos Terminados

Este caso de uso permite al Administrador, Jefe de Bodega y Confeccionista registrar y administrar los productos terminados tras finalizar el proceso de confección. Incluye funciones CRUD (Crear, Leer, Actualizar, Eliminar) para mantener actualizado el inventario de productos listos para entrega, venta o distribución.

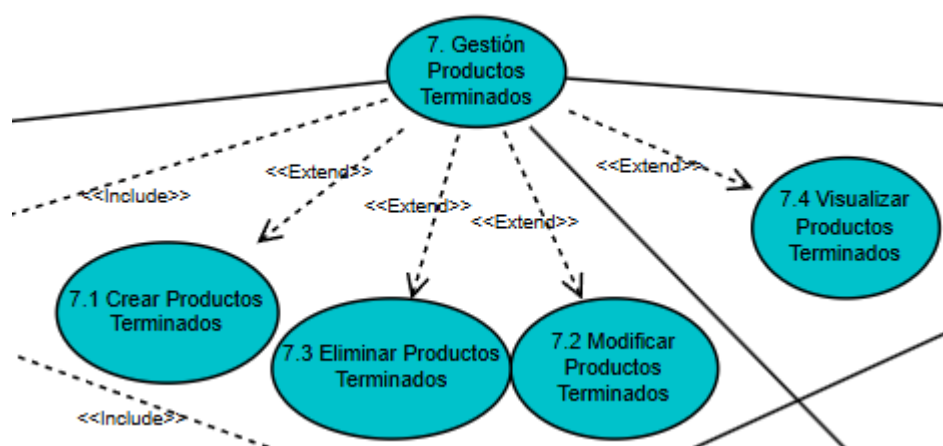
Precondiciones:

- El usuario debe haber iniciado sesión con un rol autorizado.
- El producto debe haber pasado por el proceso de confección y control de calidad (si aplica).

Flujo Principal (Escenario Exitoso):

1. El usuario accede a la sección de Productos Terminados.

2. El sistema muestra la lista de productos registrados (nombre, cantidad, fecha de finalización, ubicación, lote, etc.).



3. El usuario puede:

- Crear un nuevo registro de producto terminado al finalizar una orden.
- Consultar los productos disponibles en inventario.
- Actualizar la información del producto (por ejemplo, cantidad, ubicación, estado).
- Eliminar un registro si es necesario (según permisos).

4. El sistema guarda los cambios y actualiza el inventario correspondiente.

Código:

```
class ProductosTerminados
{
    private $idProductos;
    private $Nombre_Producto;
    private $Cantidad_Disponible;
    private $Descripcion;
    private $Fecha_Entrada;
    private $Fecha_Salida;
    private $idmateria_prima;
    private $Estado;
    private $status;
    private $pdo;

    public function __construct()
    {
        try {
            $this->pdo = new Database;
        } catch (PDOException $e) {
            die($e->getMessage());
        }
    }

    public function getAll()
    {
        try {
            $strSql = "SELECT * FROM productos_terminados pt
                        JOIN estados e ON pt.idEstado = e.idEstados
                        JOIN materia_prima mp ON pt.idmateria_prima = mp.idProducto
                        WHERE pt.status = 'IN'";
            $query = $this->pdo->select($strSql);
            return $query;
        } catch (PDOException $e) {
            die($e->getMessage());
        }
    }
}
```

```

public function newProductoTerminado($data)
{
    try {
        $this->pdo->insert('productos_terminados', $data);
        return true;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getProductoTerminadoId($idProductos)
{
    try {
        $strSql = "SELECT pt.*, e.Estados AS EstadoNombre, mp.Nombre AS
MateriaPrimaNombre
        FROM productos_terminados pt
        JOIN estados e ON pt.idEstado = e.idEstados
        JOIN materia_prima mp ON pt.idmateria_prima = mp.idProducto
        WHERE pt.idProductos = :idProductos";
        $arrayData = ['idProductos' => $idProductos];
        $query = $this->pdo->select($strSql, $arrayData);
        return $query;
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

public function editProductoTerminado($data)
{
    try {
        $strWhere = 'idProductos = ' . $data['idProductos'];
        $this->pdo->update('productos_terminados', $data, $strWhere);
    } catch (PDOException $e) {
        die($e->getMessage());
    }
}

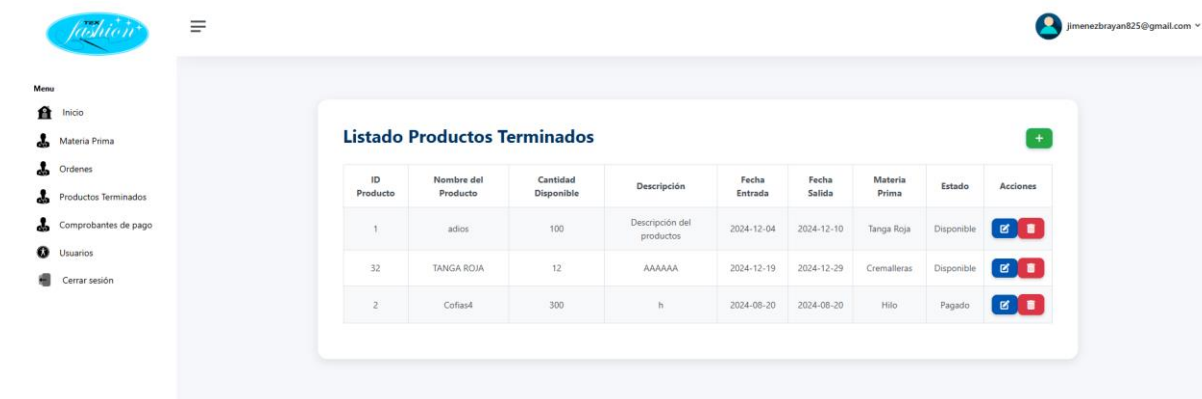
```

```
public function deleteProductoTerminado($idProductos)
{
    try {
        $strWhere = 'idProductos = :idProductos';
        $data = ['status' => 'OUT', 'idProductos' => $idProductos];
        $this->pdo->update('productos_terminados', $data, $strWhere);
        return true;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getMateriasPrimas()
{
    try {
        $strSql = "SELECT idProducto, Nombre FROM materia_prima WHERE status =
'IN'";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}

public function getEstados()
{
    try {
        $strSql = "SELECT idEstados, Estados FROM estados";
        $query = $this->pdo->select($strSql);
        return $query;
    } catch (PDOException $e) {
        return $e->getMessage();
    }
}
}
```

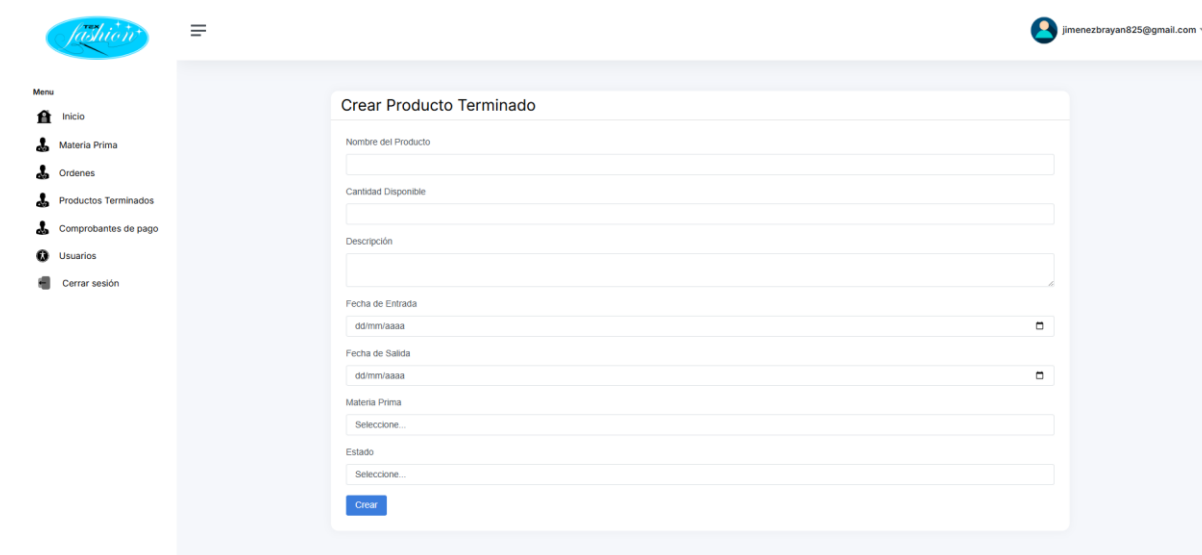
Img Interfaz



Listado Productos Terminados

ID Producto	Nombre del Producto	Cantidad Disponible	Descripción	Fecha Entrada	Fecha Salida	Materia Prima	Estado	Acciones
1	adios	100	Descripción del productos	2024-12-04	2024-12-10	Tanga Roja	Disponible	 
32	TANGA ROJA	12	AAAAAA	2024-12-19	2024-12-29	Cremalleras	Disponible	 
2	Cofias4	300	h	2024-08-20	2024-08-20	Hilo	Pagado	 

Crear Productos Terminados



Crear Producto Terminado

Nombre del Producto

Cantidad Disponible

Descripción

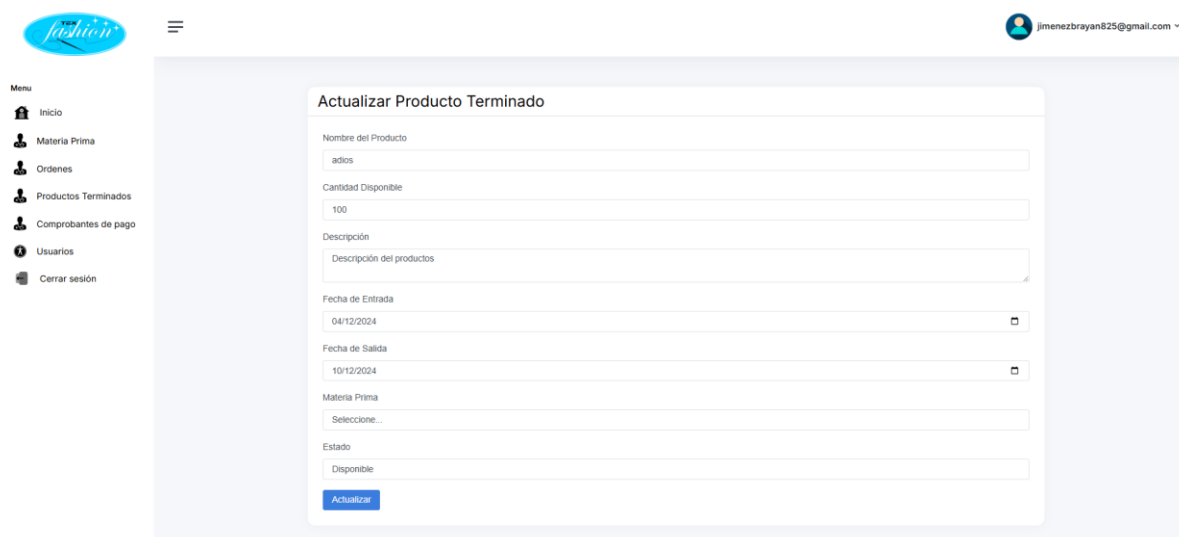
Fecha de Entrada

Fecha de Salida

Materia Prima

Estado

Modificar Producto



7. API

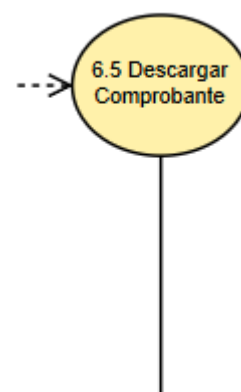
Este caso de uso describe cómo un usuario autenticado puede interactuar con una API que incluye un chatbot para atención básica y consultas, específicamente con el fin de **consultar detalles de sus facturas y descargarlas en formato PDF** u otro formato disponible. El sistema responde a solicitudes RESTful y devuelve respuestas estructuradas (JSON, archivos, etc.).

Precondiciones:

- El usuario debe estar autenticado con un token válido (por ejemplo, JWT).
- Debe existir al menos una factura registrada a nombre del usuario en el sistema.

Flujo Principal (Escenario Exitoso):

1. El usuario realiza una solicitud a la API de chat (POST /api/chat) con una consulta como "¿Puedo ver mi última factura?".
2. El sistema interpreta la solicitud y redirige la intención a la función de consulta de factura.
3. El usuario envía una solicitud a la API (GET /api/facturas) para listar sus facturas disponibles.



4. El sistema devuelve una lista de facturas asociadas al usuario (en formato JSON).

5. El usuario selecciona una factura (por ID) y envía una solicitud a GET
`/api/facturas/{id}/descargar`.
6. El sistema genera o recupera el archivo (PDF, XML, etc.) y lo devuelve como
descarga.